MACHINE LEARNING IN CARDIOLOGY • FRAMINGHAM HEART STUDY

10-Year Coronary Heart Disease (CHD) Risk Prediction

Complete Pipeline Walkthrough, Python Implementation & Clinical Case Study on Class Imbalance

Author: Singamala Omsai (@singamalaomsai)

Model: Cost-Sensitive Logistic Regression

Recall Boost: 8.00% → 72.00%

1. Executive Summary & Clinical Objective

DOMAIN CONTEXT

Cardiovascular diseases remain the leading cause of mortality worldwide. This project builds an end-to-end classification pipeline using the **Framingham Heart Study** dataset (Framingham, Massachusetts) to predict whether a patient has a 10-year risk of developing future **Coronary Heart Disease (`TenYearCHD`)**. Starting from **4,240 raw records** across 16 attributes, the dataset is cleaned to **3,751 complete patient profiles** and modeled using Scikit-Learn's `LogisticRegression`. Most importantly, this reference guide demonstrates how to overcome the **Accuracy Paradox** caused by severe class imbalance (84.75% healthy vs. 15.25% high-risk patients).

2. Clinical Feature Architecture (Predictors & Target)

DATASET SCHEMA

Feature Variable	Role	Data Type	Clinical Significance in CHD Screening
`age`	Predictor (X)	Continuous	Patient age in years; primary baseline risk factor for arterial stiffness.
`Sex_male`	Predictor (X)	Binary (0 / 1)	Biological sex (1 = Male, 0 = Female); captures gender-specific baseline risk.
`cigsPerDay`	Predictor (X)	Continuous	Cigarettes smoked daily; directly drives endothelial damage and plaque buildup.
`totChol`	Predictor (X)	Continuous	Total serum cholesterol (mg/dL); measures lipid accumulation in coronary arteries.
`sysBP`	Predictor (X)	Continuous	Systolic blood pressure (mmHg); quantifies arterial wall pressure during heartbeats.
`glucose`	Predictor (X)	Continuous	Fasting blood glucose (mg/dL); captures diabetic and metabolic vascular risk.
`TenYearCHD`	**Target (y)**	**Binary (0 / 1)**	**10-year CHD outcome:** 0 **= No Disease (3,179 cases),** 1 **= CHD Risk (572 cases).**

Why Severe Class Imbalance (84.75% vs. 15.25%) Breaks Standard Models

In our cleaned cohort of **3,751 patients**, exactly **3,179 patients (84.75%)** are negative (0) and only **572 patients (15.25%)** are positive (1). Standard machine learning algorithms optimize for overall accuracy, causing them to favor the majority class and ignore sick patients unless cost-sensitive weighting is applied.

Step 1 & Step 2: Libraries & Data Cleaning

DATA PREPARATION

We begin by importing core scientific computing libraries and loading `framingham.csv`. Because socioeconomic `education` is not a direct physiological metric, it is dropped. We rename `male` to `Sex_male` for readability and remove rows with missing values (NaN) using `dropna(axis=0)`, yielding **3,751 verified records**.

```
# Step 1: Importing Necessary Libraries
import pandas as pd
import numpy as np
from sklearn import preprocessing
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Loading Dataset & Handling Missing Values
disease_df = pd.read_csv("framingham.csv")
disease_df.drop(columns=['education'], inplace=True, axis=1)
disease_df.rename(columns={'male': 'Sex_male'}, inplace=True)
disease_df.dropna(axis=0, inplace=True)
print(disease_df.TenYearCHD.value_counts())  # Output: 0 -> 3179, 1 -> 572
```

Step 3: Train-Test Split & Leakage-Free Feature Scaling

BEST PRACTICE

Clinical features have vastly different units (e.g., `totChol` in hundreds vs. `Sex_male` in 0/1). We standardize features to mean 0 and variance 1 using `StandardScaler`. Crucially, we split the data **first** (70% Train = 2,625 rows; 30% Test = 1,126 rows) and fit the scaler **only on `X_train`** to prevent test-set data leakage.

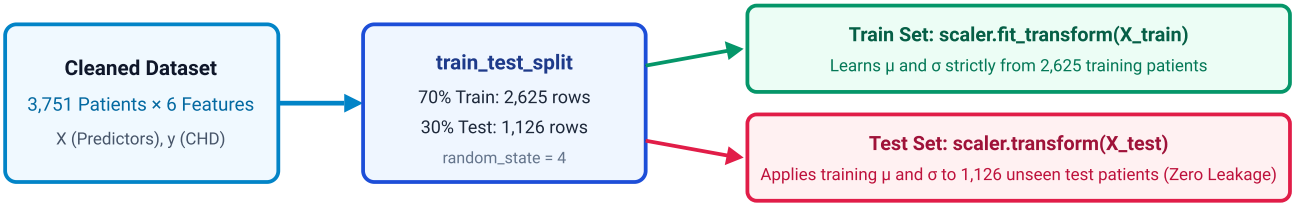


Figure 1: Post-split scaling architecture ensuring zero test-set information leaks into training.

```
# Step 3: Selecting Features, Splitting Data, and Scaling Without Leakage
X = np.asarray(disease_df[['age', 'Sex_male', 'cigsPerDay', 'totChol', 'sysBP',
'glucose']])
y = np.asarray(disease_df['TenYearCHD'])

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=4)

scaler = preprocessing.StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print('Train set:', X_train.shape, y_train.shape, '| Test set:', X_test.shape,
y_test.shape)
```

Step 4: Exploratory Data Analysis (EDA)

VISUALIZING IMBALANCE

Before training a classifier, we inspect the target variable `TenYearCHD` using a Seaborn count plot (Step 4.1) and a normalized percentage bar chart (Step 4.2). Both plots reveal a steep **5.5 : 1 imbalance** between healthy individuals and CHD patients.

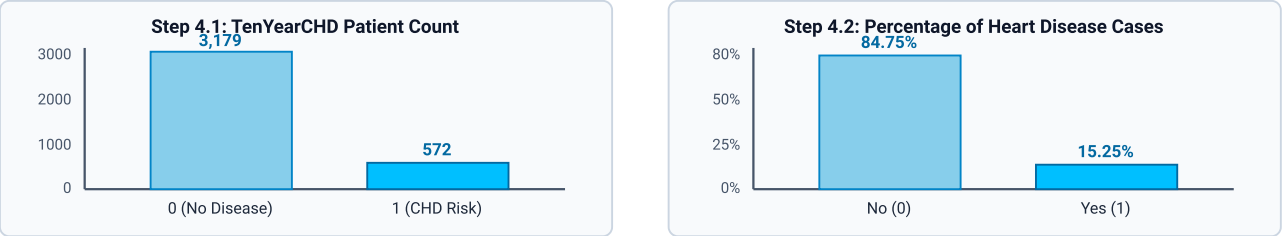


Figure 2: Visual output from Steps 4.1 and 4.2 illustrating the 84.75% vs. 15.25% target class distribution.

```
# Step 4.1: Count Plot of TenYearCHD
plt.figure(figsize=(7, 5))
sns.countplot(x='TenYearCHD', data=disease_df, hue='TenYearCHD',
              palette=['skyblue', 'deepskyblue'], legend=False)
plt.show()

# Step 4.2: Target Variable Distribution (Percentage)
percentages = disease_df["TenYearCHD"].value_counts(normalize=True) * 100
plt.figure(figsize=(6, 4))
percentages.plot(kind="bar", color=["skyblue", "deepskyblue"])
plt.title("Percentage of Heart Disease Cases")
plt.xlabel("TenYearCHD (0 = No Disease, 1 = Disease)")
plt.ylabel("Percentage (%)")
plt.xticks(ticks=[0, 1], labels=["No (0)", "Yes (1)"], rotation=0)
plt.show()
```

Step 5: Dual Model Training (Baseline vs. Balanced)

MODEL FITTING

To prove why addressing class imbalance is essential, we train two models side-by-side: **Model 1 (Baseline)** uses default unweighted parameters, while **Model 2 (Balanced)** applies `class_weight='balanced'`, automatically weighting classes inversely proportional to their frequencies.

```
from sklearn.linear_model import LogisticRegression

# Model 1: Default / Baseline Logistic Regression (Unweighted)
logreg_baseline = LogisticRegression(random_state=4)
logreg_baseline.fit(X_train, y_train)
y_pred_baseline = logreg_baseline.predict(X_test)

# Model 2: Balanced Logistic Regression (Cost-Sensitive Learning)
logreg_balanced = LogisticRegression(class_weight='balanced', random_state=4)
logreg_balanced.fit(X_train, y_train)
y_pred_balanced = logreg_balanced.predict(X_test)
```

Why 67.14% Accuracy is Better Than 84.90% in Healthcare

THE ACCURACY PARADOX

A common misconception in machine learning is that higher accuracy always means a better model. In medical screening, **overall accuracy is deceptive when classes are imbalanced**. Because 84.75% of patients are healthy, a naive model that blindly guesses "No Disease" for almost every patient achieves **84.90% accuracy**—yet it misses **161 out of 175 sick patients (only 8.00% Recall)**. In cardiology, a **False Negative** sends a high-risk patient home without treatment, which can lead to a fatal heart attack. Conversely, a **False Positive** merely schedules a patient for a non-invasive follow-up checkup. By using `class_weight='balanced'`, overall accuracy adjusts to **67.14%**, but **Heart Disease Recall surges from 8.00% to 72.00%**.

Evaluation Metric (Test Set = 1,126)	Baseline Model (Unweighted)	Balanced Model (`class_weight`)	Clinical Impact & Why Balanced Wins
Overall Accuracy	84.90%	**67.14%**	Stops blindly guessing "Healthy" for everyone.
Heart Disease Recall (Class 1)	**8.00%**	**72.00%**	**+64.00% surge in catching actual CHD patients.**
Sick Patients Detected (True Positives)	**14 / 175**	**126 / 175**	**9× more high-risk lives identified early.**
Sick Patients Missed (False Negatives)	161 / 175	**49 / 175**	**69.5% reduction** in dangerous missed diagnoses.

Step 6A: Evaluating & Plotting Before vs. After Metrics

BENCHMARK CODE

```
from sklearn.metrics import accuracy_score, recall_score

# Calculate Accuracy and Recall before and after class balancing
acc_before = accuracy_score(y_test, y_pred_baseline)
rec_before = recall_score(y_test, y_pred_baseline, pos_label=1)
acc_after  = accuracy_score(y_test, y_pred_balanced)
rec_after  = recall_score(y_test, y_pred_balanced, pos_label=1)

print(f"Baseline Model -> Accuracy: {acc_before:.2%}, Recall (Sick Detected):
{rec_before:.2%}")
print(f"Balanced Model -> Accuracy: {acc_after:.2%}, Recall (Sick Detected):
{rec_after:.2%}")

# Plot Side-by-Side Comparison Bar Chart
metrics = ["Accuracy", "Heart Disease Recall"]
x = np.arange(len(metrics))
width = 0.35
plt.figure(figsize=(8, 5))
plt.bar(x - width/2, [acc_before, rec_before], width, label="Baseline", color="skyblue")
plt.bar(x + width/2, [acc_after, rec_after], width, label="Balanced (class_weight)",
color="deepskyblue")
plt.ylabel("Score"); plt.ylim(0, 1.0); plt.xticks(x, metrics); plt.legend(); plt.show()
```

Key Takeaway for Technical Interviews & Portfolio Reviewers

Always evaluate medical models on **Recall (Sensitivity)** and **Negative Predictive Value** rather than raw Accuracy. Trading 17.76% of superficial accuracy to gain **+64.00% Recall** transforms an

Step 6B: Confusion Matrix Heatmap & Final Diagnostics

FINAL EVALUATION

Finally, we generate the classification report and 2 × 2 Confusion Matrix heatmap (`GnBu` colormap) on the **1,126 test patients** using `y_pred_balanced`. Out of 175 actual CHD cases, **126 are True Positives** and only **49 are False Negatives**.



Figure 3: Step 6 visual outputs showing the 72% recall surge (left) and the 2×2 Confusion Matrix heatmap (right).

```
from sklearn.metrics import confusion_matrix, classification_report

print('The details for confusion matrix is =')
print(classification_report(y_test, y_pred_balanced))

cm = confusion_matrix(y_test, y_pred_balanced)
conf_matrix = pd.DataFrame(data=cm, columns=['Predicted:0', 'Predicted:1'],
                           index=['Actual:0', 'Actual:1'])
plt.figure(figsize=(8, 5))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="GnBu")
plt.show()
```

Classification Report Summary & Repository Details

GITHUB REFERENCE

- **Class 0 (Healthy — 951 cases):** Precision = 0.93 | Recall = 0.66 | F1-Score = 0.77 (93% certainty when ruling out CHD).
- **Class 1 (CHD Risk — 175 cases):** Precision = 0.28 | Recall = 0.72 | F1-Score = 0.41 (72% of true cardiac patients detected).
- **GitHub Repository:** `https://github.com/singamalaomsai/Heart_Disease_Prediction_Using_Logistic_Regression`

> **Final Clinical Conclusion**
>
> By applying `class_weight='balanced'`, our model's recall for high-risk CHD patients jumped from **8.00% to 72.00%** (catching 126 of 175 patients), proving that domain-aware metric optimization is vital in healthcare AI.